JUNE 2026

Fence Stain Simulator

End-to-End Technical Walkthrough & Project Report

A complete, plain-English walk-through of how the Fence Stain Simulator is built — from scraping tens of thousands of fence photos, through AI-assisted labelling and a two-phase deep-learning model, to the live browser experience your customers use at ninjafencestaining.com — with the exact numbers, costs, and decisions behind each step.

LIVE TOOL

ninjafencestaining.com/fence-staining-color-simulator

Prepared for Ninja Fence Staining · by TechnoTaau · DINOv3 ViT-L/16 segmentation

Contents

Executive Summary

The Fence Stain Simulator is the tool a homeowner uses on ninjafencestaining.com to upload a photo of their fence, have the software automatically find the fence boards, and preview any stain colour on those boards in seconds — before spending a dollar on materials or labour. This report is the full story of how that tool is built, written for the client rather than for engineers, but without skipping any of the moving parts.

There are, in plain terms, three things going on behind the screen. First, an enormous amount of preparation work that the customer never sees: we collected, cleaned, and hand-checked roughly **33,000 fence and non-fence photos**, and turned them into pixel-perfect training labels. Second, we trained a large, modern AI segmentation model — a **DINOv3 ViT-L/16** network with several specialised add-ons — to look at any photo and decide, pixel by pixel, "this is wooden fence" versus "this is not." Third, we wrapped that model in a fast, private, browser-based experience: the photo is sent to a GPU server only for the one detection step, and everything else (colour picking, staining, cleaning, downloading) happens instantly on the user's own device.

A few headline facts the rest of this report supports in detail:

- **The data.** 21,414 fence ("positive") photos and 12,009 deliberately-chosen look-alike ("negative") photos were scraped from 13 sources, de-duplicated, quality-filtered, and catalogued into a single 33,423-image master list. Every image carries its source, search query, and (where available) its licence.
- **The labels.** Each image was auto-labelled by a two-model AI pipeline (Grounding DINO + Segment Anything 2.1), quality-scored, and the uncertain cases were hand-refined by a human using Segment Anything 3. During that human review, about **8,000 images the AI had called "fence" were corrected to "not fence,"** which is exactly the kind of correction that lifts a model's real-world accuracy.
- **The model.** A DINOv3 ViT-L/16 backbone, a ViT-Adapter, a Mask2Former decoder, and a learnable refinement head with depth guidance — roughly **half a billion trainable parameters**. It is trained in two phases (512-pixel, then 1024-pixel).
- **The training cost.** Training ran on rented data-centre GPUs for a total spend of **$220** at **$1.14 per GPU-hour**, which works out to about **193 GPU-hours — roughly eight days** of continuous GPU time, all-in (including bandwidth).
- **Where it stands.** We have completed **24 of a planned 120 training epochs (20%)** of phase one. At this early checkpoint the **staining** preview is already close — you're *almost* satisfied with it — while the **cleaning** result is **not there yet** and needs dedicated work (see the roadmap). That is exactly what you'd expect at a fifth of the planned training: there is enormous remaining headroom — ~96 more phase-one epochs and the entire second phase are still ahead, and the validation accuracy was **still climbing, not plateauing,** when we paused.
- **How it is served.** The model runs on **Google Cloud Run** on an NVIDIA L4 GPU, currently in "scale-to-zero" mode — it costs almost nothing when idle but takes 30–60 seconds to wake up on the first request after a quiet period. This report prices the alternative (keeping it warm 24/7) and the other hosting paths you could choose.
- **The browser app.** A single web page (also packaged as a WordPress plugin) does the upload, talks to the server, and then runs a deep client-side pipeline — confidence-based blending, fifteen-plus cleanup filters, WebAssembly-accelerated maths, and a small on-device AI "gate" that blocks obvious non-fence uploads before they ever reach the paid server.

The short version: the foundations are enterprise-grade and the current model is already useful, but it is an early checkpoint of a much larger training plan. The path to "absolute best quality" is clear and is laid out near

the end of this document.

1. What the product is, and the pipeline at a glance

The product

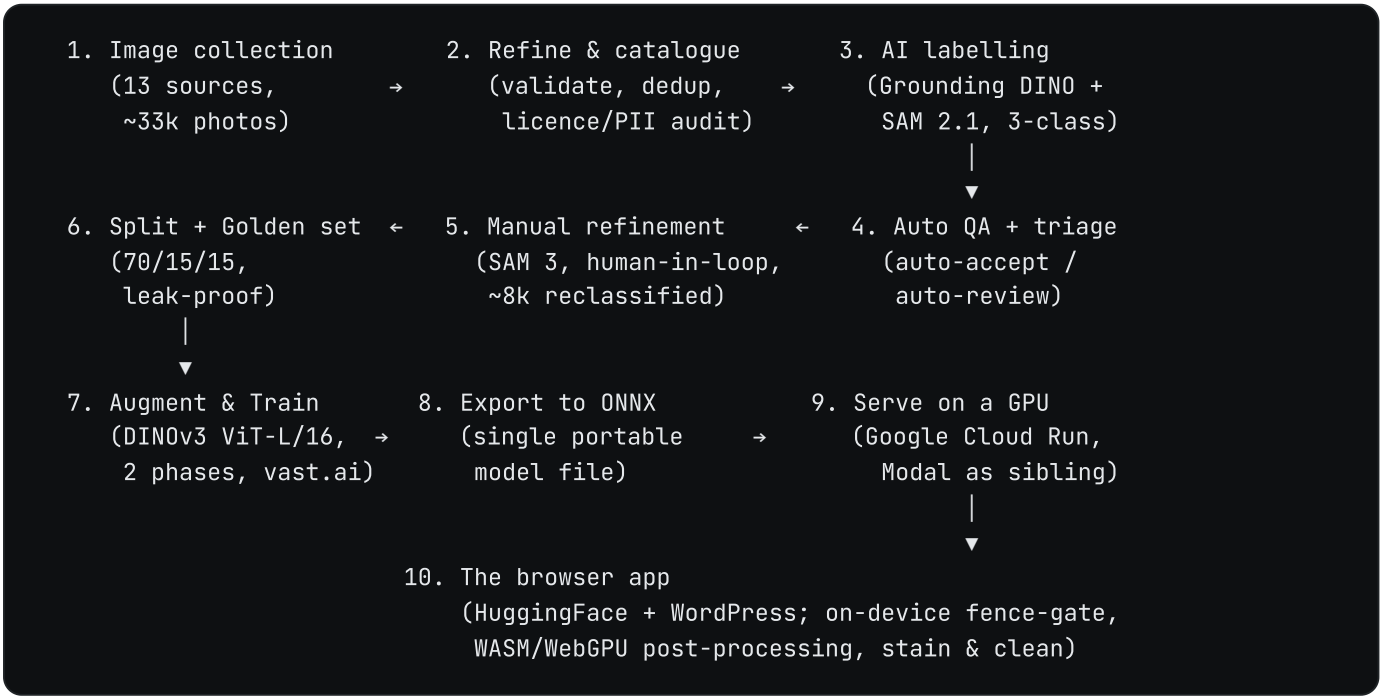
From the customer's point of view, the Fence Stain Simulator is dead simple:

1. **Upload** a photo of a fence (drag-and-drop or pick a file, up to 10 MB).
2. **Pick a colour** from 19 curated stains across three families (General, Semi-transparent, Semi-solid) and set the opacity.
3. **Apply Stain** — the app finds the fence and recolours it in the chosen stain; changing the colour or opacity afterwards re-renders the preview instantly.
4. **Clean Fence** — alternatively, see what the existing weathered wood would look like pressure-washed and restored.
5. **Download** the result as an image.

Everything that feels instant — changing colours, opacity, switching between Clean and Stain — happens locally in the browser. The only step that talks to a server is the one-time "find the fence" detection, and even that photo is processed in memory and discarded; it is never stored.

The pipeline behind it

Getting from "an idea" to "a working tool" took a long, disciplined pipeline. Here is the whole thing end to end:



The rest of this report walks through each of those boxes in order, with the real numbers, the engineering decisions, and the trade-offs. Two sections at the end are written specifically for your decisions: the **roadmap to maximum quality** (what "finishing" the model looks like) and the **hosting cost analysis** (warm vs. cold vs. your own server).

A note on accuracy and honesty in this report

Every figure in this document was read directly out of the project's own files — the configuration files, the training logs, the model metadata, the deployment scripts, and the data manifests — rather than from memory or from the older, shorter summary that existed before. Where the project's own files disagree with each other (and a few do, because they were written at different times), this report uses the most authoritative source and notes the discrepancy plainly. A handful of internal documents are now out of date relative to the shipping product; those are flagged where relevant so they don't mislead anyone later.

2. Step 1 — Collecting the images (the scraper)

An AI segmentation model is only ever as good as the photos it learns from. So the first real piece of work was building a tool that could gather a very large, very varied library of fence photographs — and, just as importantly, a library of things that *look* like fences but aren't. This is the `data_scraper/` subsystem, and it is a genuine piece of production software, not a throwaway script.

Why "negatives" matter as much as "positives"

It is obvious why you need photos of fences. It is less obvious — but just as important — that you need photos of *non*-fences that share the fence's most confusing feature: repetitive wooden slats. A deck, a pergola, wooden stairs, a slatted bench, barn siding, a wooden privacy screen, even a close-up of a wooden table — all of these can fool a naive detector into shouting "fence!" So the project deliberately built two corpora: a **positive** set of real wooden fences, and a **negative** set engineered to contain exactly the hard look-alikes. Teaching the model what *not* to stain is what keeps it from colouring a customer's deck or their neighbour's brick wall.

Where the photos come from

The scraper pulls from **13 different sources**, grouped by how they are accessed:

Type	Sources
Keyed photo APIs	Google Custom Search, Pexels, Unsplash, Pixabay, Flickr (Creative-Commons-only)
Keyless public APIs	Wikimedia Commons, Reddit
Browser-driven scrapers	Google Images, Bing Images, DuckDuckGo, Pinterest, Houzz (real Chromium via Playwright)
Direct gallery scraper	30 hand-picked fence-and-staining company websites

Each source is handled respectfully and correctly. The licensed photo APIs (Pexels, Unsplash, Pixabay) grab a sensible web-resolution rendition rather than the full 20–40 MB original, which is faster and kinder to the service. Flickr is restricted to Creative-Commons licences so the imagery is safe to reuse. Wikimedia requires a polite "who is this and how do I contact you" identifier in every request, which the scraper provides. Reddit can use proper OAuth2 authentication for a higher rate limit. The company-gallery scraper is clever but simple: most fence-company sites are WordPress, so their gallery pages link directly to full-size images, and a small "strip the thumbnail size suffix" trick recovers the originals without needing a browser at all.

A point of full disclosure for the record: the Google Images scraper (`pw_google`) is documented internally as a "Terms-of-Service-risky, emergency fallback only" source, yet it was in fact enabled and contributed several thousand images to both sets. That is noted here because it matters for licensing diligence (see §3).

The engineering that keeps it reliable

What separates this from a weekend script is everything it does to stay correct and resumable while pulling tens of thousands of images over flaky networks:

- **Adaptive rate limiting.** Every host has its own request budget. When a server replies "slow down" (HTTP 429/503), the scraper reads the suggested wait time and benches that host — but never for more than two minutes, so one rude server can't stall everything.
- **Per-host circuit breakers.** If a host fails five times in a row it is "tripped" for a minute, then probed once to see if it has recovered. This stops the scraper from hammering a dead endpoint.
- **Proxy rotation** is available (off by default) for sources that block data-centre IPs.
- **Two-tier de-duplication.** Every image is hashed two ways: an exact byte-for-byte SHA-256 (catches literal copies) and a *perceptual* hash (an 8×8 "dHash" that captures what the image looks like). The perceptual hashes are stored in a specialised data structure (a "BK-tree") so the scraper can ask "is anything visually within a hair's breadth of this new image?" in a fraction of a second, even against tens of thousands of stored hashes. The similarity threshold is a Hamming distance of 5. This is what stops the same fence photo — which appears on Pexels, Pinterest, and three company sites — from entering the dataset five times.
- **A quality gate.** Before an image is accepted it must be between 800×600 and 8000×8000 pixels, have a sensible aspect ratio, be 40 KB–25 MB, and actually decode as a valid image. Nine stock-photo watermark domains (Shutterstock, Getty, iStock, etc.) are blocked outright. The image decoder is hardened against "decompression bomb" attacks.
- **A content filter** drops anything whose title or page mentions NSFW, violence, or "clipart/wallpaper/illustration" keywords.
- **An optional Google Vision verification gate** (described below).
- **Full resumability.** All state — every hash seen, every URL tried, every failure, and how far it got on each search query — lives in a local SQLite database, so a run that crashes or is stopped picks up exactly where it left off. The positive corpus was assembled over roughly **16 separate scraper sessions**.

The Google Vision quality check

For the positive set, an optional but powerful second opinion was used: after download, an image could be sent to Google Cloud Vision, which returns a list of labels ("Fence," "Wood," "Yard," "Wall," "Garden," etc.). The image is accepted only if a fence-related label scores above a confidence threshold. To control cost (Vision bills about $1.50 per 1,000 images), the scraper samples deterministically — the same image always gets the same accept/reject decision and is never re-billed on a re-run — and it is permissive on any error, so a Vision outage never throws away good photos. The threshold was set to **0.6 for the normal positive run** and deliberately relaxed to **0.4 for the "hard positives"** run, on the sound logic that a genuinely difficult fence photo (heavily occluded, oddly lit) will score low on the fence label, and a strict cutoff would discard exactly the hard examples the model most needs.

The three collection runs

The same engine was pointed at three different jobs, each with its own configuration file:

Run	Target	Vision	Notes
Positives (`scraper.yaml`)	17,000	On @ 0.6	144 base fence queries + AI-expanded queries + 22 cedar-specific + 30 company galleries
Hard positives (`scraper_hard_positive.yaml`)	22,000 (top-up)	On @ 0.4	191 curated "hard" queries (occlusion, damage, harsh light, mid-staining), same database as positives
Negatives (`scraper_negative.yaml`)	12,000	**Off**	Separate folder & database; 226 look-alike queries; Wikimedia & company sites disabled to avoid accidentally pulling in fences

What we actually ended up with

The on-disk reality, verified directly from the files and databases:

- **Positives:** 21,414 images on disk. Top contributors: Pexels (≈5,000), Bing (≈4,200), Wikimedia (≈3,500), Google (≈3,000), Houzz (≈1,900). The top search queries are exactly what you'd hope — "cedar fence," "cedar privacy fence," "cedar horizontal slat fence," "bamboo fence."
- **Negatives:** 12,009 images on disk. Led by Google (≈2,700) and Houzz (≈2,500). Its queries are a careful mix of obvious non-fences (forest trails, mountains, beaches) and the deliberate hard negatives (wooden lattice panels, louvered doors, deck boards, log-cabin walls, horizontal blinds).
- **Combined raw harvest: 33,423 images** after the cross-set clean-up described in the next section.

Each image is stored with a filename that encodes its own provenance (`source__query__hash.jpg`) and a line in a master `metadata.jsonl` catalogue recording where it came from, what query found it, its dimensions, and its hashes.

Where each set's images actually came from (counts verified from the catalogues):

Source	Positive	Negative
Pexels	5,166	1,759
Bing (Playwright)	4,166	532
Wikimedia Commons	3,519	—
Google (Playwright)	3,038	2,740
Houzz (Playwright)	1,859	2,545
Unsplash	1,160	1,654
Pixabay	1,014	1,629
Company galleries	935	—
Pinterest (Playwright)	817	1,150
Total	**21,674**	**12,009**

(Reddit, Flickr, and DuckDuckGo are fully implemented but were left disabled in the live runs, so they contributed nothing — worth knowing if those sources are ever needed later.)

The search-query strategy. Beyond a hand-written set of **144 base queries** organised into nine themed groups, the positive run added AI-generated query variations and 22 cedar-specific phrasings, and the hard-positive run used a separate set of 191 deliberately difficult queries. The nine base groups are a good window into how carefully the variety was engineered:

Group	Queries	Targets
Wood styles	26	cedar, pine, redwood, picket, shadowbox, board-on-board, stockade, split-rail
Non-wood styles	9	vinyl, chain-link, wrought-iron, aluminium (as edge cases)
Scenes	16	backyard, front yard, garden, suburban, along driveway/sidewalk
Occlusion	24	behind plants, vines, bushes, snow, tall grass
Humans / animals	15	pets, children, people staining/installing
Distractors	15	next-to shed/pergola/deck/gazebo
Scales	10	close-up, macro, aerial, distant, long perspective
Conditions	14	golden hour, rain, frost, fog, dramatic shadow
Variations	15	broken, rotting, gates, half-finished, missing boards

A small honesty note for the record. On the positive side, three different tallies don't perfectly agree — the catalogue lists 21,674 entries, the scraper's database holds 21,518, and 21,414 files actually sit on disk. This is the normal drift you get when files are pruned after the catalogue was last written. The number that matters — what physically exists and feeds the next stage — is the 21,414, and the negative side reconciles perfectly at 12,009. It is mentioned only so nobody is later confused by the small gaps.

3. Step 2 — Refining, validating and cataloguing the data

Raw scraped data is always a little dirty. Before a single photo is allowed near the model, it goes through one disciplined pass — the `prepare_dataset.py` script — whose entire job is to produce one trustworthy master catalogue (`manifest.jsonl`) that the rest of the pipeline can rely on absolutely.

The integrity check

Every image is opened and tested in four escalating stages: does the file exist and have non-zero size; compute a fresh SHA-256 over the actual bytes (deliberately *not* trusting any hash already recorded, so corruption is caught); a quick structural check; and finally a full pixel decode to prove the image isn't a half-downloaded grey smear. Truncated JPEGs are rejected rather than silently accepted. This runs across all CPU cores at once, and if interrupted it throws away partial results rather than writing a half-finished catalogue. On the real run, **33,674 files were scanned and zero were found corrupt** — a testament to the quality gate already applied during scraping.

Cross-set de-duplication (and why direction matters)

Because the positive and negative sets were scraped separately, the same image can land in both. The script finds every positive whose byte-hash matches a negative and **removes it from the positives, keeping the negative copy.** The reasoning is deliberate and documented: these collisions are almost always a stock API serving a loose match, or a non-wood fence that was wrongly scraped as a positive — and in both cases the negative label is the more trustworthy one. This removed exactly **251 images**, every one logged in `removed.jsonl` with the reason "cross-set-dup." After this step the catalogue holds **33,423 images**.

Cataloguing by subcategory

Every surviving image is tagged with a fine-grained subcategory derived from its original search query — about 20 positive subcategories (e.g. `style_cedar`, `style_wood`, `occlusion`, `damaged_construction`, `lighting`, `scene_context`) and a dozen negative ones (e.g. `neg_masonry`, `neg_pure_random`, `neg_nonwood_fence`, `neg_siding`). This taxonomy is what later lets us (a) split the data *evenly* across every category, (b) over-sample the rare hard cases during training, and (c) report accuracy broken down by category so we can see exactly which situations the model finds hard. The largest positive buckets are cedar fences (≈4,800), generic wood fences (≈3,900), and general fence scenes; the largest negative bucket is "pure random" non-fence imagery.

The audit reports that ship alongside the data

Three governance reports are produced and kept with the dataset — these are real enterprise due-diligence artifacts:

- **Resolution report.** Every image is sorted into resolution tiers (Ultra ≥2048 px, HD, Standard, Low). Everything with a shorter edge of at least 1024 px — **20,700 images (62%)** — forms the "HQ subset" reserved for the high-resolution second training phase.
- **Licence audit.** Every source is classified SAFE (Pexels/Unsplash/Pixabay — 12,367 images, 37%), SAFE-with-attribution (Wikimedia — 3,519 images, 10.5%), or RISKY (search-engine and company-site scrapes — 17,537 images, 52.5%). The audit recommends, for a fully clean commercial deployment, training on the

15,886-image SAFE subset (`manifest_safe.jsonl`). This is an important strategic option and is revisited in the roadmap section.

- **PII scan.** A face-detection pass (Google Vision) over the images most likely to contain people, flagging any significant faces for GDPR/CCPA review.

> **Licensing is a real decision, not a footnote.** Just over half the current training corpus comes from sources whose reuse rights are ambiguous (browser-scraped search results and company galleries). For training an internal model this is the industry norm; for a commercial product it is worth a conscious decision. The project already produced the clean 15,886-image subset that sidesteps the issue entirely. Whether to retrain the production model on only the SAFE subset is a business/legal call covered in the roadmap.

4. Step 3 — AI-assisted labelling and manual mask refinement

This is the stage that turns a folder of photos into the pixel-perfect "answer key" the model learns from. For each image we need to know *exactly which pixels are wooden fence.* Drawing that by hand for 33,000 images would take months, so the project uses a two-layer system: a fully automatic labelling pipeline that produces a first-pass mask for every image, and a human-in-the-loop layer that only ever touches the images the automatic pass was unsure about.

The three-class idea (the clever bit)

The labelling uses a deliberately-designed **three-class scheme**:

- **Class 1 —** `fence_wood` : the only thing the simulator will ever stain.
- **Class 2 —** `not_target` : a "decoy" / absorber class for wood-like things that are *not* a stainable fence (wood siding, decks, furniture, vinyl/metal fences).
- **Class 0 — background**: everything else.

The trick is in the priorities. When the AI ambiguously matches something as both "cedar fence" and "wood siding," the `not_target` class is given *higher* priority and wins the pixel — so it does **not** get stained. From the app's point of view the output is still simply "fence vs. not fence," but giving the AI an explicit "wood-like-but-not-fence" category dramatically cuts the false positives that a pure yes/no scheme would be forced into.

For completeness: a richer **24-class** labelling schema was actually built first (`configs/annotation_schema.yaml`) — with fine-grained fence materials, several occluder types, and scene-context classes across five tiers, each with its own detection prompts and thresholds. It was deliberately collapsed down to today's 3-class scheme because, from the stainer's point of view, the only distinction that ultimately matters is fence-wood versus everything-else, and the simpler scheme labels faster and more reliably. The 24-class schema remains in the repository as the superseded original.

Stage 1 — the automatic pipeline

For each image, in order:

1. **Scene pre-filter (CLIP).** A zero-shot CLIP classifier (`openai/clip-vit-base-patch32`) checks whether the photo is even an outdoor fence scene or an out-of-distribution one (an interior, a document, a product shot). OOD images aren't dropped, but they're flagged for review and their confidence is halved.
2. **Detection (Grounding DINO).** `IDEA-Research/grounding-dino-base` , an open-vocabulary detector (~680M parameters), is given a long list of text phrasings ("wooden fence," "cedar fence," "wood plank fence," …) and draws boxes around what it finds. It detects on the original *and* a mirrored copy for a few extra points of recall, and if it finds nothing it retries once with relaxed thresholds.
3. **Segmentation (SAM 2.1).** Each box is handed to `facebook/sam2.1-hiera-large` (Segment Anything 2.1), which turns the box into a pixel-accurate mask. SAM is asked for three candidate masks per box and the best is kept — which is exactly what preserves the gaps between fence slats. Each mask is then snapped to the true image edges with a guided filter.
4. **Fusion.** All the per-object masks are merged into one class map by priority, with two protective rules: a confident fence is shielded from being eaten by overlapping "occluder" detections, and a `not_target`

decoy can only overwrite a fence pixel if it beats the fence's confidence by a clear margin. This single margin is what stops a real cedar fence being "absorbed" by a spurious "wood siding" match.

5. **Scoring & flagging.** A confidence score and a set of quality flags are computed; any flag at all routes the image to the human review queue.

Up to three artifacts are written per image — the machine-readable class mask, a black-and-white preview (white only where the stain target is), and a colourised overlay for human review. The overlay is generated by default but was pruned from the full 33k set to save disk space (it's retained for the QA and golden-set subsets).

Stage 2 — automatic triage (so humans only see hard cases)

Rather than make a person look at all 33,000 images, two bulk scripts pre-decide the obvious ones. `auto_accept_positives.py` stamps high-confidence, clearly-wood-fence positives as "reviewed" without changing their masks — but *only* if the image is already a positive, its subcategory is on a trusted whitelist, and its coverage and confidence are in a sensible band. `auto_review_negatives.py` does the mirror image for known negatives, wiping their masks to empty and stamping them reviewed. Whatever survives triage is, by construction, the genuinely ambiguous set — and that is what a human sees first.

Stage 3 — manual refinement with Segment Anything 3

The polished review tool (`manual_refine_sam3.py` , ~2,300 lines) loads Meta's newest **Segment Anything 3** image model with interactive editing. A reviewer clicks to add or remove regions, drags a box, cycles through SAM's candidates, or paints with a manual brush. When they save, an edge-refinement cascade (a DenseCRF pass, then a guided filter, then morphology) cleans up the boundary — but each stage is only kept if it doesn't erode the mask too much. The reviewer only ever sees images that genuinely need attention, ordered worst-first, so human time scales with the number of *hard* cases, not with the size of the dataset.

The most important single fact in the whole data pipeline

During this human review, the class balance of the dataset **changed dramatically.** The automatic pipeline had labelled 21,414 images as fence and 12,009 as not-fence. After human review, the final catalogue (`manifest_final.jsonl`) is **13,328 fence and 20,095 not-fence** — meaning roughly **8,000 images the AI confidently called "fence" were corrected to "not fence" by a human.** That is an enormous, deliberate correction. It is exactly the kind of label cleaning that separates a demo-quality model from a production one, and it is the reason the deployed model is as conservative (in the good sense) about *not* staining the wrong things as it is. Any older document that still quotes "21,414 fence images" is describing the pre-review state.

`export_final.py` is the script that merges the immutable automatic results with the human override log to produce that final catalogue. Every row's class provenance is stamped via that review log (`class_source: manual_review`) — all 33,423 images pass through review, with the ~8,000 genuine class flips recorded there too — so the provenance of every label is auditable.

5. Step 4 — The Golden Set (the quality yardstick)

Before training a model, you need a ruler you trust. The **Golden Set** is that ruler: a small, hand-curated, hand-masked benchmark used to answer the only question that matters during development — "did this change make the model better or worse?"

It is built by `select_golden_set.py`, which picks **100 images drawn exclusively from the held-out test split** (so they are never seen during training), stratified across **20 subcategories** so the hard cases are all represented — the common cedar and wood fences alongside a guaranteed slot for the genuinely difficult corners (extreme occlusion, harsh lighting, reflections on water, painted fences, odd angles, weather extremes). The selection is fully reproducible (fixed random seed, and the source split is hashed so the set is invalidated if the underlying data ever changes). A senior annotator then hand-masks all 100 images at pixel quality — budgeted at three to five hours of careful work.

The Golden Set has four documented jobs, each with a numeric bar:

- **Auto-label quality check** — the automatic pipeline should score above **0.70 IoU** against these hand masks.
- **Inter-annotator agreement** — a second human re-masking the same images should agree above **0.90 IoU**.
- **Per-version regression guard** — every model update is scored against the Golden Set, and any drop is treated as a regression.
- **Deployment sign-off** — the Golden Set is the sanity floor that should always sit above the test-set average.

("IoU," Intersection-over-Union, is the standard segmentation accuracy score: of all the pixels that are *either* truly fence *or* predicted fence, what fraction are both? 1.0 is perfect; for this kind of natural-image fence segmentation, scores in the 0.5–0.7 range on the full diverse test set are strong, and higher on the cleaner subsets.)

6. Step 5 — Splitting into Train / Validation / Test

The 33,423-image catalogue is divided into three slices that the model is never allowed to confuse:

Split	Count	Purpose
Train	23,394 (70%)	What the model learns from
Validation	5,013 (15%)	Checked every epoch to track progress and pick the best model
Test	5,016 (15%)	Held back, untouched, for the final honest score

Two pieces of engineering make this trustworthy, and both are the kind of thing that quietly separates a credible result from an inflated one:

Stratification. The split is balanced by (class × subcategory), so every one of the ~50 (class × subcategory) strata is proportionally represented in all three slices. The model never gets surprised at test time by a category it barely saw in training.

Leak-proofing against near-duplicates. This is the subtle one. Even after de-duplication, the dataset contains *near*-duplicate photos (the same fence shot from a slightly different angle, or the same image at two resolutions). If a near-duplicate of a test image sneaks into the training set, the model can "memorise" it and the test score becomes a lie. To prevent this, the splitter clusters all visually-similar images (using the same perceptual-hash technique from scraping) and forces every cluster to travel *together* into a single slice. Before writing anything, it asserts two things that "cannot silently fail": the three slices sum exactly to the input count, and no image appears in more than one slice. The test set is then made **read-only on disk** so it cannot be accidentally overwritten.

The whole split is reproducible from a fixed seed (42) and recorded in an audit file (`split_info.json`) that captures the exact command, the catalogue's hash, and the resulting counts. A parallel **high-resolution split** (train 14,529 / val 3,087 / test 3,084, all from the 20,700 HQ images) is prepared for the second training phase. And a parallel set of **mask manifests** links each image to its pixel-mask file, so the training code can load image and answer-key together.

> **One naming clarification for the folder.** The `dataset/` directory contains several files with similar names because each is a checkpoint in this pipeline; a full disambiguation table is in §15. The short version: `manifest_final.jsonl` (post-human-review) is the current source of truth, the splits under `dataset/splits/` are what training actually reads, and the various `manifest_hq*`, `manifest_safe`, and `.bak` files are the high-res subset, the licence-clean subset, and backups respectively.

7. Step 6 — Augmentation (teaching the model to generalise)

A model that only ever sees each training photo once tends to memorise rather than understand. Augmentation fixes this by showing the model thousands of randomly-perturbed variations of every image, so it learns the *concept* of "wooden fence" rather than the specific pixels of any one photo. The augmentation here is unusually fence-specific rather than generic.

Geometric and "distance" variety. Random flips, rotations, perspective warps, and a "distance scaling" block that either zooms in on a fence, shrinks-and-pads it (to simulate a far-away fence), or does a **boundary-aware crop** centred on a fence edge — so the model spends its learning budget where it matters: the fence-vs-grass and fence-vs-sky boundaries.

Photometric variety. Strong, deliberate colour, brightness, contrast, and sharpness jitter (applied to most images), plus simulated harsh sun, shadows, lens flare, blur, noise, and JPEG compression. The heavy colour jitter is intentional: it breaks the model's lazy habit of assuming "brown = fence," forcing it to rely on shape and structure.

Mixing. Two images are blended together (CutMix), and four images are tiled into one (Mosaic), so the model learns to handle scenes with multiple things going on.

The two custom fence augmentations. These are the clever ones:

- **Copy-paste occluder.** Real cut-out objects — people, cars, and the like — from a pool of 621 cutouts (421 extracted from the COCO dataset plus 200 procedurally-generated occluders) are pasted *in front of* the fence, and the answer-key mask is correctly carved away wherever the occluder sits. This teaches the model to handle the branch, the parked bike, the garden hose in front of the fence.
- **Hard-negative wood paste.** Wooden *non*-fence objects (planks, deck patches, garden beds) are pasted into the background, with the mask left unchanged — explicitly teaching "wood texture alone is not a fence."

Balanced sampling. During each pass over the data, rare-but-important subcategories (occlusion, complex backgrounds, damaged fences) are over-sampled so the model doesn't ignore the long tail of edge cases just because they're uncommon.

The augmentation is aggressive in phase one (512 px) for maximum generalisation and deliberately gentler in phase two (1024 px) to preserve the fine boundary detail that the higher resolution exists to capture. (All of this lives in the data-loading code, `tools/dataset.py`; the aggressive-vs-gentle split and the RandAugment strength are driven by the `train.*` keys in `configs/phase1.yaml` and `configs/phase2.yaml`, so the exact augmentation regimen is traceable to those files.)

8. The model architecture, in full

This is the heart of the system and the part you asked to have spelled out completely — the backbone, the decoder, the ViT-Adapter, the refinement head, the MiDaS depth model, the EMA, and everything else. The model is large and modern: a six-stage pipeline of roughly **half a billion trainable parameters.** Each stage exists for a concrete reason, explained below in plain terms followed by the precise settings.

First, the single most important clarification, because it is a common point of confusion in the project's own files: **the production model uses the DINOv3 ViT-L/16 backbone** (`facebook/dinov3-vitl16-pretrain-lvd1689m`), *not* the larger "H+" variant. The H+ model appears only as a leftover default in one code file and as a "use this if you run out of memory" comment — it is **not** what was trained or deployed. The shipped model's own metadata confirms ViT-L/16 at 512-pixel input. Wherever an older note says "H+" or quotes a ~1-billion-parameter count, it is stale.

Here is the full stack, in the order a photo flows through it:

Stage 1 — The backbone: DINOv3 ViT-L/16

The backbone is the "eyes" of the model — the part that looks at the raw pixels and produces a rich numerical description of what's in the image. We use **DINOv3 ViT-L/16** from Meta, a Vision Transformer with about 300 million parameters, pre-trained by Meta on 1.7 billion images using self-supervised learning (no human labels). Starting from such a strong pre-trained backbone is what lets us reach good accuracy with "only" tens of thousands of fence images instead of millions.

Key specifics:

- **Vision Transformer, Large, patch size 16.** The image is cut into a grid of 16×16-pixel patches; a 512-pixel input becomes a 32×32 grid of patch "tokens." (This is exactly why the input is 512, not 518 — 512 ÷ 16 is a clean 32, whereas the older DINOv2 used patch size 14 and needed 518. Getting this wrong caused visible banding artifacts once, and the app's own configuration now carries an explicit warning never to use 518, which is DINOv2's value.)
- **Hidden dimension 1024, 24 transformer layers, 4 "register" tokens** (DINOv3's mechanism for cleaner attention).
- **Multi-block aggregation.** Instead of using only the final layer's output, the model fuses the last **6 layers** with a learnable weighted sum — dense pixel prediction benefits from combining several depths of features.
- **Fully trainable, but gently.** The whole backbone is fine-tuned on fence data, but at a learning rate ten times lower than the rest of the network, so it adapts to fences without forgetting the powerful general vision it learned from 1.7 billion images.

Stage 2 — The ViT-Adapter

A plain Vision Transformer only produces one coarse scale of features (one feature per 16×16 patch). Pixel-accurate fence masks — especially thin pickets — need fine, multi-scale detail. The **ViT-Adapter** (Chen et al., ICLR 2023) is the bridge: a small convolutional network runs alongside the transformer and, at four points during the transformer's processing, the two exchange information through "deformable attention." The result is a proper feature pyramid at four scales (strides 4, 8, 16, 32) instead of one.

Specifics: a Spatial Prior Module (a small CNN, 64 base channels) produces the four scales; four interaction stages (8 attention heads, 4 sampling points each) are spaced evenly through the 24 transformer layers (at layers 6, 12, 18, 24); the injection starts at zero strength and ramps up during training so it never disturbs the pre-trained backbone at the start. The deformable-attention maths is implemented in pure PyTorch (no custom GPU kernels), which is part of why the model exports cleanly to a portable format later.

Stage 3 — The pixel decoder (Mask2Former / MSDeformAttn)

The four-scale feature pyramid is refined by a **multi-scale deformable-attention pixel decoder** — the real Mask2Former pixel decoder, 12 layers deep, 8 heads, operating across three scales at once. Its job is to mix information across scales and locations so that, for example, a faint picket at the top of the frame is interpreted in the context of the strong fence structure below it. It produces a high-resolution feature map used to draw the actual masks.

Stage 4 — The Mask2Former transformer decoder

This is where the masks are actually predicted. A **Mask2Former-style decoder** holds **64 learnable "query" vectors** — think of them as 64 hypotheses, each of which learns to grab one coherent region — and refines them over **15 decoder layers** (512-dimensional, 8 heads). Each layer uses Mask2Former's two defining features:

- **Masked attention:** each query is restricted to attend only inside its own current best guess of its region, which sharpens masks and speeds convergence.
- **Global tokens:** the backbone's summary tokens (the CLS token plus the 4 register tokens) are made available to every layer as whole-image context.

The decoder also emits a prediction after *every* layer (16 in total) for "deep supervision" during training, which gives the earlier layers a direct learning signal.

Stage 5 — The refinement head (the boundary specialist)

The Mask2Former mask is good but slightly soft at the edges. A learnable **UNet3+-style refinement head** sharpens it, and runs for **3 iterations**, each time taking its own previous output as input and improving it. It is fed the original RGB image, the coarse mask, features from the pixel decoder, a "vertical position" hint (sky is up, ground is down), and a depth map (next stage). It is configured at 96 channels with 4 blocks, and carries a small zoo of specialist sub-heads, all of which exist to make boundaries crisp and to suppress mistakes:

- an **edge head** that predicts the fence outline directly;
- a **signed-distance head** that learns how far each pixel is from the nearest boundary (sub-pixel sharpness);
- **full-scale deep supervision** side outputs;
- a **PointRend** module that re-examines the 4,096 *most uncertain* pixels through a small dedicated network — concentrating effort exactly on the ambiguous boundary pixels;
- a **Classification-Guided Module (CGM)** — a whole-image "does this picture contain any fence at all?" classifier that, at inference time, multiplicatively suppresses the mask on no-fence images. This is a major weapon against false positives on negatives.

Stage 6 — The MiDaS depth teacher

This is the part many people are curious about. The refinement head is given a **monocular depth map** — an estimate of how far away each pixel is — produced by a frozen, pre-trained **MiDaS / DPT model** (`Intel/dpt-hybrid-midas`). Depth is a strong cue for fences: a fence is usually a roughly-flat surface at a consistent distance, distinct from the deep background behind it and the foliage in front. The depth model is **frozen** (never trained — it is a fixed "teacher"), run without gradients, and its output is normalised to a 0–1 relative-depth map and handed to the refinement head as an extra input channel. There's even a deliberate safeguard that keeps the depth model in evaluation mode across the whole multi-day training run so its internal statistics never drift.

EMA — the "smoothed" copy of the model

Throughout training, the system maintains an **Exponential Moving Average** of the model's weights — a second copy that is a smoothed, running average of the model as it trains (decay 0.999, i.e. roughly a 1,000-step window). Training is noisy and the weights oscillate from batch to batch; the EMA copy is calmer and almost always generalises better. **Validation is run on the EMA copy, and the EMA copy is what gets saved as "best" and ultimately deployed.** So the model serving customers is the smoothed average, not the jittery instantaneous one. (In phase two, the EMA also acts as a "teacher" that the live model is gently pulled toward — a self-distillation technique that further stabilises fine-tuning.)

Memory engineering: gradient checkpointing

A half-billion-parameter model with a 15-layer decoder and a 3-iteration refinement head is a lot to fit in GPU memory. **Gradient checkpointing** is enabled, which trades a little extra computation for a large memory saving (it recomputes intermediate values during the backward pass instead of storing them all). This is what lets the model train at a useful batch size on a single 80 GB GPU.

Putting numbers to it

Component	Setting
Backbone	DINOv3 ViT-L/16, 1024-dim, 24 layers, 4 register tokens, last-6-block weighted-sum
ViT-Adapter	64 base channels, 8 heads, 4 points, 4 interaction stages
Pixel decoder	MSDeformAttn, 12 layers, 8 heads, 3 scales
Transformer decoder	Mask2Former, 512-dim, 64 queries, 15 layers, masked attention + global tokens
Refinement head	UNet3+, 96 channels, 4 blocks, 3 iterations; edge + distance + PointRend + CGM + depth
Depth teacher	Intel/dpt-hybrid-midas (frozen)
Stabiliser	EMA (decay 0.999), gradient checkpointing
Output	Single-channel fence probability map
Trainable size	~485 million parameters (engineer's estimate; computed at runtime)

Phase one and phase two use the **identical architecture** — deliberately, so phase two can pick up exactly where phase one left off. Only the regimen differs (resolution, learning rate, augmentation strength), covered next.

9. Step 7 — Training the model on vast.ai

The two-phase plan

Training is designed in two phases that share the same architecture:

	Phase 1	Phase 2
Resolution	512 px	1024 px
Data	Full 33k split	HQ subset (≥1024 px)
Epochs (planned)	120	70
Starting point	Pre-trained DINOv3	Phase-1 best checkpoint
Purpose	Maximum data exposure	Recover fine boundary detail

Phase one trains on everything at moderate resolution to learn the concept broadly; phase two is a gentler fine-tune at high resolution to sharpen the edges that 512 pixels simply can't represent. Because the two phases use an identical network, phase two starts from phase one's best weights rather than from scratch.

The training signal (the loss)

The model is taught with a carefully-balanced combination of about **a dozen loss components**, each targeting a different aspect of mask quality. You don't need the maths, but the *shape* of it tells you how much care went in:

- **Pixel classification** (weighted cross-entropy) with three fence-friendly tweaks: a positive-class weight of 3.0 (fence pixels are rarer than background, so they count for more), a "focal" term that concentrates effort on the hard look-alike pixels, and an "online hard-example mining" term that focuses on the worst-scoring 25% of pixels each step.
- **Region overlap** losses (Dice, Tversky) that directly optimise the IoU-style overlap, tilted slightly toward recall so the model doesn't miss fence.
- **Boundary, edge, and signed-distance** losses that specifically reward crisp, correctly-placed fence outlines.
- **A Lovász term** (a direct IoU surrogate) and a **connectivity** term (discourages broken, speckled masks).
- **PointRend importance sampling** — the loss is concentrated on the most uncertain boundary pixels.
- **The CGM classifier loss** — teaches the whole-image "is there any fence here?" gate.
- **Deep supervision** — every intermediate decoder layer gets its own learning signal.

The exact phase-1 weights, for the record:

Loss component	Weight	What it rewards
Cross-entropy (BCE)	1.0	Correct per-pixel fence/not call (pos-weight 3.0, focal γ 2.0, hard-25% mining)
Dice	1.0	Region overlap
Tversky	0.5	Overlap, tilted to recall (α 0.6 / β 0.4)
Boundary	0.4	Pixels near the true fence edge
Lovász	0.25	Direct IoU surrogate
Connectivity	0.02	Discourages broken/speckled masks
Deep supervision	0.3	Every decoder layer gets a signal
PointRend	(12,544 pts)	Concentrates loss on uncertain boundary pixels (distinct from the refinement head's 4,096 inference-time points)
Edge head	0.8	A crisp predicted outline (edge pos-weight 4.0)
CGM classifier	1.0	The whole-image "is there any fence?" gate
Boundary-distance	0.3	Sub-pixel distance-to-edge regression

(The refinement head's own copies of BCE/Dice/boundary/Tversky are applied at half weight, plus extra terms for the iterative refinement and the UNet3+ side heads.) Many of these weights carry visible "tuning history" in the configuration — they were adjusted across numerous rounds (the files literally label them "FIX A" through "FIX I" and "audit fix") to stop the model from developing bad habits like producing solid blobs instead of respecting the gaps between pickets. That iterative, evidence-driven tuning is exactly the unglamorous work that makes a model production-quality.

Optimizer and schedule

Standard, well-chosen modern settings: the AdamW optimizer; a head learning rate of 1.5e-4 and a backbone learning rate ten times lower; layer-wise learning-rate decay (earlier backbone layers learn more slowly); cosine schedule with an 8-epoch warm-up; gradient clipping for stability; mixed-precision (bf16) maths for speed and memory; and gradient accumulation to reach an effective batch size of 16 on a single GPU. There is robust crash-resilience throughout — bad batches that produce non-finite losses are skipped, out-of-memory spikes are caught and the batch retried, and the whole run can resume from a checkpoint with its random state intact.

The hardware and the cost

Training ran on a **rented cloud GPU instance via vast.ai**, configured as:

- **1× NVIDIA A100-SXM4-80GB GPU**
- **128 GB system RAM**
- **Intel Xeon Platinum CPU**

The economics, derived directly from the spend:

> **Total spend: $220.** At the **$1.14 per GPU-hour** on-demand rate, that is **$220 ÷ $1.14 ≈ 193 GPU-hours — about 8 days of continuous GPU time.** This figure is "all-in": it includes the bandwidth and storage charges that sit on top of raw compute, so the pure-compute slice is slightly under 193 hours.

It's worth being transparent about where those ~193 hours went, because not all of it was the final clean training run. Cross-checking the training logs: the clean, deployed phase-one run was about **3.9 hours per epoch** for **24 epochs ≈ 94 hours**. Training was restarted about four times (a normal reality of multi-day cloud runs — a code fix here, an interruption there), and counting the re-run epochs the logs show roughly **146 hours** of actual training compute. The remaining time was consumed by the earlier model generation (the DINOv2 experiment described later), repeated model exports, and — significantly — the sheer time spent **downloading the multi-gigabyte model files off the rented box** over a throttled connection, which the project worked around with elaborate multi-stream download tooling. All of it bills GPU rental while the box is up, which is why the money-derived ~193 hours is the honest total and an epoch-only estimate would undercount.

Where training stands today

This is the most important status fact in the report, stated plainly:

> **Phase one is 24 of a planned 120 epochs complete — 20%. Phase two (a further 70 epochs) has not started.** The deployed model is the **epoch-24 checkpoint**, and it is the best checkpoint produced so far.

The validation accuracy climbed steadily and was **still rising, with no sign of plateauing,** when the run was paused:

Epoch	Validation IoU
1	0.4244
8	0.4603
15	0.4787
18	0.4855
23	0.4918
24 (deployed)	**0.5014**

Where the model is already strong, and where it needs work

Because the data was tagged by subcategory, training logs the accuracy *per category* — which is far more useful than a single average, because it tells us exactly where to aim the remaining work. These are the logged per-subcategory validation IoUs from around epoch 18 (the overall has since improved to 0.50 at epoch 24), rounded and best read as *indicative of relative strength* rather than exact:

Category	Val IoU	Read as
Non-fence: random scenes, siding, shutters, masonry	0.93 – 1.00	**Excellent** — it almost never stains the wrong thing
Non-fence wooden look-alikes (decks, panels, dividers)	0.88 – 0.96	**Excellent** — the hard-negative work paid off
Scene-context fences (typical backyard shots)	~0.67	**Strong** — the bread-and-butter case
Mildly occluded fences	~0.66	**Good**
Cedar fences	~0.57	**Good**
Heavily occluded fences	~0.58	**Good**
Generic wood fences	~0.51	**Fair**
Damaged / under-construction fences	~0.38 – 0.41	**Needs work**
Extreme scale (very close / very distant)	~0.38	**Needs work**
Ambiguous "general fence"	~0.27 – 0.30	**Weak**
Painted / unusually-coloured fences	~0.26	**Weak**
Harsh / dramatic lighting	~0.23	**Weak**
Complex / cluttered backgrounds	~0.21	**Weak**

The shape of this is encouraging for a model only a fifth of the way through training: it is **outstanding at not staining the wrong things** (every non-fence category sits in the 0.9s), solid on ordinary fences, and weakest on the genuinely hard cases — harsh lighting, painted fences, cluttered scenes, damage. Those weak buckets are precisely the targets for the "more hard data" and "finish training" steps in the roadmap (§16). A model this conservative about false positives is a good foundation; the remaining job is lifting the hard positives, which more training and more targeted data directly address.

Two honest caveats that belong in front of the client:

1. **These are validation numbers, not final test numbers.** Because the run was paused mid-stream, the automatic held-out *test* evaluation (which runs only at the very end, with test-time augmentation and full post-processing) never executed. The model is promising — the **staining** previews are already close to what you want — but a single headline "test IoU" number doesn't exist yet because the run hasn't reached its finish line, and the **cleaning** result still needs work (addressed directly in the roadmap, §16).
2. **The curve was still going up.** An IoU of 0.50 at epoch 24, climbing, on a plan that budgeted 120 epochs plus a whole second high-resolution phase, means the model you're seeing today is an *early* checkpoint of a much larger plan — not a finished product hitting its ceiling. That is the good-news framing of the roadmap in §16.

Checkpoints and resumability

Every save writes several files: the full training state (for resuming), a stripped weights-only file (for deployment), the EMA copy, and periodic snapshots. Each checkpoint also embeds full provenance — the git commit, the library versions, the exact GPU, and a timestamp — so any result is reproducible and auditable. Resuming is a single command and picks up at the exact epoch, optimizer state, and random seed where it stopped. **In other words, finishing the training is not a restart — it is a resume.**

10. Step 8 — Exporting the model to ONNX

A model trained in PyTorch can't be served efficiently as-is. The export step (`tools/export_onnx.py`) converts the trained checkpoint into **ONNX**, an open, framework-neutral format that any inference runtime can load. This is the bridge between "research artifact" and "thing on a server."

The export is thoughtful in ways that matter for production:

- **It bakes the calibration into the model.** The trained model's temperature calibration is folded directly into the exported graph, so the server doesn't have to (and must not) re-apply it. The output is a clean probability map in the 0–1 range under a single output named `mask_prob`. (For the current epoch-24 model the baked temperature is 1.0 — an identity, i.e. no-op — and the per-subcategory thresholds aren't set yet, because the calibration pass only runs at the *end* of a finished training phase, which hasn't happened (see §16). The mechanism is in place and will apply real calibration the moment training completes.)
- **It ships the refined output.** By default it exports the full refinement-head output (the production-quality boundaries), not the coarser intermediate mask.
- **It records the full input contract.** A sidecar JSON file is written next to the model documenting exactly how to feed it: a 512×512 RGB image, normalised with standard ImageNet statistics, in the precise channel order — plus the model's training config, provenance, recommended post-processing, and the parity-check result. This sidecar is what makes the model genuinely portable; every server that hosts it reads the same contract.
- **It verifies itself.** After export, it runs the same input through both the original PyTorch model and the new ONNX model and checks they agree to within a tiny tolerance (5e-3). The earlier (epoch-18) export passed this with a maximum difference of 0.00003 — essentially identical.

The resulting model is two files: a small **13.9 MB graph** (13,902,638 bytes) and a **2.45 GB weights file** (2,451,570,688 bytes — the weights exceed ONNX's 2 GB single-file limit, so they live in an external `.onnx.data` file). The two are inseparable and travel together everywhere the model is deployed.

> **One honest caveat.** The currently-deployed epoch-24 export's *self-check* didn't get to finish — ONNX Runtime hit a memory-allocation error on the laptop running the check (the parity check runs on the CPU, so this was a host-memory limit, not a model problem or a GPU issue). The model file was written correctly before the check ran, and the architecture is identical to the epoch-18 export that *did* pass cleanly, so it is almost certainly correct. (The same crash also meant the epoch-24 export's fresh sidecar JSON wasn't written — the only sidecars currently on disk are the epoch-18 and epoch-23 backups.) Re-running the export on a larger machine is a sensible, cheap thing to tick off: it both validates parity and regenerates the live sidecar.

11. Step 9 — Hosting the model on a GPU server

The trained model is too large to run inside a web browser (2.45 GB), so it runs on a GPU server and the browser talks to it over the internet for the one detection step. The project built this twice: first on **Modal**, then migrated the live service to **Google Cloud Run**. Both are described because both exist in the codebase, and the Modal version remains a perfectly good fallback.

Modal.com — the first production host

Modal is a serverless GPU platform. The project's Modal app (`app_dinov3.py`) wraps the ONNX model in a small web service exposing two endpoints: a health check and the `/detect` endpoint that takes an uploaded photo and returns the mask. It runs on Modal's cheapest GPU (an NVIDIA **T4**, 16 GB), scales to zero when idle, and keeps a warm container for 10 minutes after the last request.

The single most interesting piece of engineering here — and it carried directly over to Cloud Run — is getting the GPU to actually engage. A lean container image ships only the GPU *driver*, not the CUDA *libraries* the model needs; if those libraries aren't found at exactly the right moment, the model silently falls back to the CPU and runs about **25× slower**. The solution is precise: install the CUDA libraries as Python packages, set the library search path *before* Python starts (setting it from inside Python is too late — the dynamic linker only reads it once at launch), and explicitly pre-load the eight critical libraries at import time. The container logs which GPU providers are active on startup so a CPU-fallback regression is immediately visible. This is the kind of detail that's invisible when it works and catastrophic when it's missing.

There is also an older Modal app (`app.py`) that served the previous-generation DINOv2 model on CPU, and a local development server (`local_server.py`) that runs the exact same `/detect` protocol on a developer's own machine for testing without redeploying. A standalone diagnostic tool (`diagnose_mask.py`) can replay the entire browser pipeline against the model to pinpoint which post-processing filter is responsible if a mask ever comes out wrong.

Google Cloud Run — the current production host

The live service today runs on **Google Cloud Run** on a single NVIDIA **L4** GPU. It is a plain, robust FastAPI container — deliberately boring, which is what you want for production. Its exact specification, read straight from the deploy script:

Setting	Value
GPU	1× NVIDIA L4 (24 GB), single-zone (no zonal redundancy)
CPU / RAM	4 vCPU / 16 GiB
Concurrency	4 requests per instance
Scaling	**min-instances 0**, max-instances 1 (scale-to-zero)
Region	us-central1
Execution	gen2, CPU always allocated, 300 s request timeout
Access	Public endpoint, port 8080

The model files are **baked into the container image** rather than downloaded at startup, a deliberate choice that shaves 5–15 seconds and the egress cost off every cold start. Preprocessing is verified to exactly match training (512 px, bilinear resize, ImageNet normalisation) — a segmentation model fed even slightly-wrong inputs degrades badly, so this parity is treated as a hard contract. The endpoint returns the mask as a compact grayscale PNG (~30–100 KB) where each pixel's brightness is the model's confidence.

The cold-start behaviour (the crux of the hosting question)

Because the service is set to **scale-to-zero** (min-instances 0), it genuinely turns itself off when nobody is using it — which is why it costs almost nothing at idle. The trade-off is the **cold start**: the first request after an idle period has to boot a fresh GPU instance, initialise CUDA, and load the 2.45 GB of weights into GPU memory, which takes roughly **30–60 seconds**. Every request after that, on the now-warm instance, is fast (sub-second to a couple of seconds). When traffic stops and the idle window elapses, the instance is torn down and the next visitor pays the cold-start penalty again.

To hide this from users, the browser app fires a "wake-up" ping at the server the moment the page loads, so the container is already warming up in the background before the visitor clicks "Apply Stain," and it shows a friendly "the simulator is starting up" message if the first request is slow.

One operational detail worth confirming: the exact idle window before an instance is torn down is left at Cloud Run's platform default — it isn't pinned in the deploy script (whereas the Modal sibling pins a 10-minute warm window). How *often* a cold start recurs therefore depends on that default, so it's worth confirming and, if useful, pinning it explicitly to tune the balance between idle cost and cold-start frequency.

This cold-vs-warm trade-off is exactly the decision laid out, with costs, in §17.

12. Step 10 — The browser app: how the JavaScript actually works

This section answers, in detail, your request to explain exactly how the web inference works — detection, staining, cleaning, preprocessing, the passes, the post-processing, WebGPU, WebAssembly, and the on-device fence gate. The current app is a single page, `index4_dinov3.html` (about 10,600 lines), and it is also packaged as a WordPress plugin (`wordpress/app.js`) with byte-for-byte identical logic.

The key mental model: **the browser is thin on AI and thick on cleanup.** It never runs the big segmentation model itself. It uploads a small photo, gets back a confidence map, and then spends the overwhelming majority of its code turning that confidence map into a clean, believable stained- or cleaned-fence image — entirely on the user's device.

12.1 Upload and preprocessing

When a photo is chosen, the app keeps the full-resolution original (for the final high-quality composite) but, for the server call, downscales it so its longest side is at most **1024 pixels** and re-encodes it as **JPEG at 85% quality** — typically 150–400 KB, so the upload is fast even on mobile data. Just before sending, it applies a *mild* enhancement (a touch more contrast and saturation, plus a percentile auto-levels pass) to help borderline photos detect — modest by design, so it helps detection without changing what the customer sees. Files over 10 MB are rejected up front.

12.2 The detection call and the warm-up

The app POSTs the JPEG to the Cloud Run `/detect` endpoint and receives a 512×512 grayscale PNG. It decodes that PNG and reads each pixel's value as the model's **confidence** that the pixel is fence, as a number from 0 to 1. (A historical note that matters for anyone reading the code: the configuration constant is still named `MODAL_ENDPOINT` and some comments still say "Modal," but the live URL is a Google **Cloud Run** address. The name is a leftover from the Modal-first days; the host is Cloud Run.) On page load the app pings the server's health endpoint to start it warming, exactly to mask the cold start described above.

12.3 Confidence as the blending alpha — the core idea

This is the conceptual heart of the whole app. The returned mask is **not** treated as a hard yes/no stencil. Its grayscale value *is* the model's confidence, and that confidence becomes the *opacity* of the stain at each pixel. The raw confidence is run through a "soft threshold": anything below 0.50 is forced to zero (kills faint false detections), anything above 0.85 keeps its full value, and the band between is a smooth ramp. The result is a hand-painted-looking alpha channel — razor-sharp and opaque where the model is sure, gently feathered where it's uncertain, and clean everywhere else. This is why the stain looks like it belongs on the wood instead of being a flat cut-out.

12.4 The detection cascade (up to five passes)

If the first detection comes back weak or empty, the app doesn't give up — it runs a multi-stage cascade designed to spend as little server time as possible:

1. **Free re-reads of the same result.** The returned mask is re-interpreted at progressively looser thresholds (strict → moderate → relaxed) — no new server call.
2. **One aggressive re-upload.** The photo is re-sent once with the stronger enhancement and gets the same strict → moderate → relaxed re-reads. These whole-image passes are deliberately **capped at two server calls** total to control cost and latency.
3. **Last resort: tiled inference.** Only if all of the above still fail, the image is cut into 3×2, 4×3, and 5×4 grids and each tile is detected separately to catch very small or distant fences — with whole-image gating and shape checks so a deck or wall can't sneak through. This path is intentionally the last resort because each tile is its own server call (up to ~38 across the three grids), making it far more expensive than the capped whole-image passes.

12.5 The post-processing gauntlet (~15 filters)

After thresholding, the confidence map runs through a long, *ordered* sequence of cleanup filters, each targeting a specific real-world failure mode. In order, roughly:

1. **Upscale** the 512-px mask back to working resolution (bilinear).
2. **Spatially-guided recovery** — find the very-confident "core" fence (≥0.85), grow that core outward by ~20 px to define a "fence zone," and inside that zone only, re-admit weaker pixels down to 0.40. This recovers shadowed or oddly-lit planks *without* inviting false positives elsewhere.
3. **Connected-component cleanup** — drop isolated specks smaller than a tiny fraction of the image (tuned low, because the DINOv3 model produces finer, more fragmented predictions than the old one).
4. **Vegetation / green filter** — remove pixels where green clearly dominates red and blue (leaves and bushes in front of the fence).
5. **Sky filter** — remove bright, low-saturation pixels in the top of the frame.
6. **Bark, trunk, and brick filters** — adaptive filters that compare each pixel to the fence's own average colour and remove tree bark, tree trunks, and red masonry.
7. **Per-component colour-outlier filter** — drop whole blobs whose colour is far from the main fence.
8. **A second recovery pass** — re-admit weathered pixels adjacent to confirmed fence.
9. **Building / barn-wall filters** — drop whole components whose confidence or saturation is far below the strongest fence component (a big building behind the fence).
10. **Junk-blob filter** — drop small, roundish, non-elongated blobs; real fence structure is elongated, so this catches the round false-positive specks that extreme or backlit lighting produces.
11. **Hole-fill** — fill small interior gaps (knots, nail-heads) while preserving the real gaps between pickets.
12. **Bottom-extend** — recover the low-confidence kickboard at the base of the planks.

Two of these filters (an orientation filter and a principal-axis filter) are present but deliberately **disabled**, because in practice they were carving out real fence structure — a good example of the empirical, "measure then keep or cut" discipline applied throughout. A nice engineering detail: every pixel-distance and pixel-area threshold is automatically scaled to the working resolution, so a "20-pixel" operation behaves identically regardless of the source photo's megapixels.

12.6 Apply Stain

"Apply Stain" derives a near-solid alpha from the soft mask (pixels above a low confidence threshold get full opacity; only the thin outer fringe keeps a ramp) — this is the trick that makes the stain read as *solid* rather than letting 15% of the old weathered wood bleed through. It then measures the fence's own average

lightness and saturation and blends the chosen stain with the default **"smart" blend mode**, which keeps the stain's *hue* but lets each pixel's lightness and saturation follow the wood grain — so the result tracks the real texture instead of looking like flat paint. Other modes (multiply, overlay, screen, colour) are available, and the opacity slider scales the whole effect.

There are **19 stain colours in three families** the customer can pick from:

- **General (5):** Leatherwood, Oxford Brown, Redwood, Natural Cedar, Cedar Tone.
- **Semi-transparent (5):** Chestnut, Mahogany, Pecan, Sequoia, Walnut.
- **Semi-solid (9):** Auburn, Barnwood, Black, Cape Cod Gray, Chocolate, Eucalyptus, Palomino, Sable, Slate Gray.

12.7 Clean Fence

"Clean Fence" simulates a pressure-wash-and-restore. It derives a per-image "clean wood" reference colour by averaging the brightest 35% of the fence's pixels (the sun-exposed, least-weathered planks), then does a **frequency-preserving** clean: an edge-aware guided filter smooths away dirt and algae blotches while respecting plank boundaries; a finer filter separates the wood-grain texture so it can be kept and gently re-sharpened; subtle per-plank variation is added so it doesn't look painted; and a uniform brightness lift restores the wood to the target tone while preserving its natural shadow-and-highlight variation. The result composites back onto the full-resolution original through the same feathered mask, so non-fence pixels stay perfectly sharp — and it's cached so a subsequent "Apply Stain" works on top of the restored wood.

12.8 WebAssembly — making the maths fast

All that per-pixel maths (colour-space conversions, blurs, morphology, the guided filter) would be slow in plain JavaScript. So the heavy loops are written in C and compiled to **WebAssembly** (`fsv_postprocess.c` → two `.wasm` files). The app ships two builds: a **SIMD128** build that processes four pixels at once (~2–3× faster) and a plain scalar build as a fallback; on startup it feature-detects whether the browser supports SIMD and loads the right one. The WebAssembly module implements the sliding-window dilate/erode, the masked box blur, the sRGB↔Lab colour conversions, the full guided-filter pipeline, the feathered alpha-blend, and the soft-mask threshold. Every WebAssembly call is wrapped so that if anything fails, it transparently falls back to an equivalent JavaScript implementation — the app degrades gracefully and never breaks.

12.9 WebGPU — optional GPU acceleration

There is also one **WebGPU** compute shader — a separable max-filter "dilate" — used as a secondary accelerator. The app only reaches for the GPU on the large-radius dilation operations where GPU parallelism actually pays off (radius ≥ 8 and image ≥ 250,000 pixels), and only when WebAssembly isn't available. The full acceleration ladder is WebAssembly-SIMD → WebAssembly-scalar → WebGPU → plain JavaScript, and the output is bit-identical across all four, so accuracy never depends on which path a given browser takes.

12.10 Download

The final result is exported as a high-quality JPEG, with a filename that reflects what was produced (original / cleaned / stained / cleaned-and-stained).

13. The on-device "fence gate" (blocking junk before the server)

Before a single byte is sent to the paid GPU server, the app runs *its own* small AI model, right in the browser, to answer one question: does this photo plausibly contain a wooden fence? This is the **fence gate**, and it exists for a simple reason — every server call costs money and time, so there's no point spending a GPU inference on a selfie, a plate of food, a screenshot, or a coffee cup sitting on a wooden table. If a fence doesn't clearly win, the upload is blocked on the device with a friendly "No fence detected in this photo" message, and the server is never called.

Why object *detection*, not image *classification*

This is the genuinely clever design decision. A simple image classifier (the CLIP/SigLIP style) scores the *whole image* against a concept like "wood." That fails badly on the fence problem: a wooden table, a wooden deck, and a coffee cup on a wooden surface all light up the "wood" concept just as strongly as an actual fence, because the whole-image signal is dominated by wood texture. So the gate instead uses a zero-shot **object detector**, which has to draw a *bounding box* around the thing it found. A coffee cup on a table has no fence to box; a small fence half-hidden behind pool umbrellas still produces a fence box even at 5% of the frame. That "must localise it as an object" requirement is exactly what separates "fence in a busy scene" from "wood close-up."

The model and how it runs

The gate uses **OWL-ViT** (`Xenova/owlvit-base-patch32` , ~80 MB), a zero-shot object detector, loaded through the transformers.js library from a CDN. It runs entirely in a **Web Worker** (a background thread), so the page never freezes during the one-to-three-second inference. It tries **WebGPU first** (fastest), with up to three retries on network hiccups, and falls back to **WebAssembly** if WebGPU isn't available. The model downloads once and is cached in the browser, so repeat visitors get an effectively instant gate. (Historical note: the gate was previously a heavier OWLv2 model and, before that, a SigLIP classifier; the leftover SigLIP "baked features" files in the repo are dead weight from that earlier design and can be ignored or deleted. Some code comments still say "OWLv2"/"SigLIP" but the model actually loaded is OWL-ViT v1.)

The decision rule

The gate sends two lists of phrases in a single inference: eight **fence** phrasings ("wooden fence," "cedar fence," "wooden privacy fence," …) and nine **distractor** phrasings ("wooden stairs," "wooden deck," "wooden railing," "wooden table," "wooden chair," …). It then picks the best-scoring qualifying fence box and the best-scoring qualifying distractor box, and passes the image only if the fence beats the distractor by a small margin. The thresholds (a score floor of 0.010, a minimum box area of 2% of the image, a stricter 10% area floor for distractors, and a competitive margin of 0.005) were tuned on real test images. The reason for the *relative* comparison rather than a fixed cutoff is instructive: wooden stairs were scoring *higher* on the "wooden fence" query than some real fences did — but stairs score even higher on the "wooden stairs" query, so comparing the two buckets makes the correct call where no absolute threshold could. ("Tree trunk" was removed from the distractor list because it matched almost any outdoor scene and was wrongly blocking real fences.)

Fail-open, and identical on both buttons

The gate is **fail-open** by design: any error — no model, network failure, decode problem — returns "yes, it's a fence" and lets the request through, because the cost of wrongly blocking a real customer (a lost sale) far outweighs the cost of wrongly passing one junk image (a fraction of a cent). The server-side detector remains the final authority. The gate runs **identically** on both "Apply Stain" and "Clean Fence," and it is preloaded on page idle so it's ready before the user acts.

14. Step 11 — Deployment: HuggingFace and WordPress

The same browser app is shipped two ways, both talking to the same Cloud Run backend.

Standalone static page (HuggingFace Spaces / GitHub Pages / Cloudflare). `index4_dinov3.html` is a self-contained page that can be hosted anywhere static — no server needed on the hosting side, because the only backend is the Cloud Run model endpoint. It is served from a HuggingFace Static Space (these resolve on a `.static.hf.space` subdomain) and/or GitHub Pages; a Cloudflare (Wrangler) deployment path is also available. This is the quickest way to get a shareable, public URL.

WordPress plugin (the production site). For ninjafencestaining.com the app is packaged as a proper WordPress plugin (`fence-stain-simulator.php` , v2.0.0). It registers a `[fence_simulator]` shortcode that drops the whole simulator into any page or post. The engineering here is genuinely careful: the simulator is mounted inside a **Shadow DOM** — an isolated sub-document — so the WordPress theme's CSS can never reach in and break the simulator's styling, and the simulator's styling can never leak out and break the site. The plugin even handles the messy reality of page builders (it checks Elementor's stored data for the shortcode, because the normal WordPress detection misses Elementor pages). The plugin's JavaScript is the same logic as the standalone page; it just loads into the isolated shadow document.

The folder also carries the expected packaging history — several versioned plugin `.zip` files, with the current shippable one being the most recent. These are normal release artifacts.

The R&D ladder (and why several files look like "models" but aren't)

One thing worth clarifying so nobody is misled by the folder contents: there are several older model files in the web directory — a UNet++ browser model, a SegFormer-B3 browser model, and a DINOv2-Small browser model — plus their conversion scripts and an out-of-date README that describes a "100% in-browser, no uploads" product. These are the **earlier rungs of the R&D ladder**, not the current product. The project genuinely tried four generations:

1. **UNet++** (in-browser) — the original.
2. **SegFormer-B3** (in-browser) — best IoU ≈ 0.455.
3. **DINOv2-Small** (in-browser, then moved server-side) — best IoU ≈ 0.425.
4. **DINOv3 ViT-L/16** (server-side) — the current production model, IoU 0.50 and climbing at only 20% of its training plan.

The first three were small enough to run in the browser but hit an accuracy ceiling. The current DINOv3 model is far more capable but too large for a browser (2.45 GB), which is precisely why the architecture moved to a server with the thin-client design described above. For the report's purposes, only generation 4 is the live product; the rest are the documented evolution that led to it. (One naming gotcha for anyone browsing the folder: the file `best_unetpp_v2.pth` is mislabelled — it actually holds the SegFormer-B3 weights, not UNet++; the genuine UNet++ model is `fence_model_unet_browser.onnx` .)

How the winning architecture was chosen

The jump to the DINOv3 server model wasn't a hunch. The repository contains a two-stage **genetic-algorithm search** over model designs (`configs/ga_stage1_model_search.yaml` and `configs/ga_stage2_hyperparam_search.yaml`). Stage one searches across roughly **18 candidate model families** — DINOv2-L and -G, SAM 2 encoders, EVA-02, InternImage, Swin-V2, ConvNeXt-V2, SegFormer-B5,

UNet++-B7, BEiT-v2, plus ensembles and cascades — scoring each on a composite of IoU and boundary quality, with weak candidates killed early on proxy epochs to save compute. Stage two then tunes the hyperparameters of the winning family. The production stack the simulator ships today — a DINOv3 ViT-L/16 backbone with a ViT-Adapter, a Mask2Former decoder, and a refinement head — is the lineage that search pointed to (DINOv3-L being the newer successor to the DINOv2-L family the search favoured). It's worth surfacing because it shows the architecture was the deliberate outcome of a structured comparison, not an arbitrary pick.

15. The `dataset/` folder, demystified

You specifically asked for this, because the `dataset/` folder contains several files with very similar names and it isn't obvious which is which. Here is the definitive map. The key insight: the folder is a **control plane**, not an image store — it holds the catalogues, splits, masks, and audits that define *what the data means*, while the 23 GB of actual JPEGs live elsewhere.

The manifest lineage (read left to right by pipeline stage)

File	Rows	What it is	Status
`manifest.jsonl`	33,423	The **auto-labelled baseline** catalogue (21,414 fence / 12,009 not). Immutable upstream record.	Baseline
`manifest_final.jsonl`	33,423	**Post-human-review** catalogue (13,328 fence / 20,095 not). **The phase-1 source of truth.**	**Current**
`manifest_hq.jsonl`	20,700	High-res (≥1024 px) subset of the *pre-review* manifest.	Superseded
`manifest_hq_final.jsonl`	20,700	High-res subset of the *final* manifest. **The phase-2 source of truth.**	**Current**
`manifest_safe.jsonl`	15,886	The legally-clean (SAFE + attribution) subset for commercial deployment.	Optional
`manifest.jsonl.bak`	33,423	Automatic safety backup.	Backup

The plain-English flow: scrape → `manifest.jsonl` (auto-labelled) → human review corrects ~8,000 labels → `manifest_final.jsonl` → the high-resolution slice becomes `manifest_hq_final.jsonl`. The licence audit separately produces `manifest_safe.jsonl` as a side branch.

What training actually reads

Crucially, the training configs **do not name any manifest directly** — they read the **splits**. The splits were generated *from* `manifest_final.jsonl`:

Location	Contents
`dataset/splits/{train,val,test}.jsonl`	The phase-1 splits (23,394 / 5,013 / 5,016). `test.jsonl` is read-only.
`dataset/splits/{train,val,test}_hq.jsonl`	The phase-2 high-res splits (14,529 / 3,087 / 3,084).
`dataset/splits/*_masks.jsonl`	The per-split links from each image to its pixel-mask file.
`dataset/splits/split_info.json`	The audit record (seed, command, counts, source hash).
`dataset/splits_smoke/`	A tiny 30/10/10 toy split for testing the code in seconds. Not training data.

The masks and the rest

Folder / file	What it is	Status
`annotations_v1/masks/`	The 33,423 pixel-mask answer keys (the labels).	Current
`annotations_v1/results.jsonl`	Raw automatic-pipeline output (immutable).	Current
`annotations_v1/manual_review.jsonl`	The human override log (source of the ~8,000 corrections).	Current
`golden_set/`	The 100-image hand-masked QA benchmark.	Current
`hard_negatives/wood/`	200 wooden-non-fence cut-outs for the hard-negative augmentation.	Current
`occluders/`	621 occluder cut-outs (421 from COCO + 200 procedural) for the copy-paste augmentation.	Current
`integrity.json` / `removed.jsonl`	The data-refining audit (33,674 scanned, 251 removed).	Current
`resolution_report.json` / `RESOLUTION_REPORT.md`	The resolution-tier audit (HQ subset = 20,700).	Current
`licenses_per_source.json` / `LICENSE_AUDIT.md`	The licence audit (SAFE/RISKY breakdown).	Current
`DATASHEET.md`	Dataset datasheet — **stale** (quotes pre-review counts).	Stale
`ANNOTATION_GUIDELINES.md`, `ANNOTATION_SYSTEM.md`, `ANNOTATION_CLASS_SCHEMA.md`	Annotation design/guideline docs — **partly superseded** (describe an earlier binary/Gemini or 20–25-class scheme; the production masks actually realise the 3-class background / fence_wood / not_target scheme).	Superseded
`training_set/` + `finetune_runs/`	A 100-image set for an experimental detector fine-tune that was never completed.	Experimental
`sam3_test/`, `trials/`, `_coco_cache/`	Throwaway experiments and a one-time download cache.	Artifacts

Bottom line: the current training path is `manifest_final.jsonl` → `dataset/splits/` (+ HQ, + masks) → the DINOv3 model. Everything else is the baseline, a superseded intermediate, the licence-clean option, a backup, an experiment, or an augmentation pool. The one document to *not* quote as current is `DATASHEET.md`, which still shows the pre-review class counts.

16. Current model state, future training, and the road to maximum quality

Exactly where we are

Item	Status
Production model	DINOv3 ViT-L/16, **phase-1 epoch 24**
Validation IoU	**0.5014** (still climbing)
Phase-1 progress	24 of 120 epochs (**20%**)
Phase-2 progress	**Not started** (0 of 70 epochs)
Final test evaluation	Not yet run (training paused before the finish hook)
Resume readiness	Full checkpoint saved; resuming is one command

The deployed model is an **early, healthy checkpoint of a much larger plan.** The current state is exactly what you'd expect at this stage: the **staining** preview is already *almost* where you want it, while the **cleaning** result is **not yet satisfactory**. Both of those judgements are coming from a model that has seen only a fifth of its planned phase-one training and none of its high-resolution phase-two fine-tuning — so the remaining headroom is not incremental, it is the majority of the plan, and (as explained below) the cleaning result has a second, separate lever beyond model training.

Why there is so much headroom

Three independent levers are all still almost entirely unused:

1. **~96 more phase-one epochs.** The validation curve was rising at a healthy clip and showed no plateau at epoch 24. Simply continuing phase one to its planned 120 epochs is the single highest-confidence improvement available, and it is a *resume*, not a restart.
2. **The entire phase two (70 epochs at 1024 px).** Phase two exists specifically to sharpen the fine boundary detail — the crisp edges around individual pickets — that a 512-pixel model fundamentally cannot represent. This is where "good mask" becomes "indistinguishable-from-hand-traced mask," which is exactly what makes a stain preview look photoreal.
3. **The post-training polish that hasn't run yet.** Temperature calibration, per-subcategory threshold tuning, test-time augmentation, and the final held-out test evaluation all run *at the end* of a completed phase. None has happened yet because the run is mid-stream. These typically add a meaningful, free accuracy bump on top of the trained weights.

The enterprise-grade path to absolute best quality

You asked specifically for the "perfect enterprise-grade solution for absolute best quality." Here is the honest, prioritised plan — ordered by return-on-effort, not by glamour:

Tier 1 — Finish what's already built (highest confidence, lowest risk).

- **Resume and complete phase one** (epochs 25–120). Pure resume; no new engineering. Estimated cost: at ~3.9 GPU-hours/epoch and $1.14/hr, ~96 epochs ≈ **$425–$550** of GPU time (with the same caveats about restarts/overhead that made the first 24 epochs cost what they did).
- **Run phase two** (1024 px, 70 epochs). Higher-resolution epochs are slower (likely 2–4× the per-epoch time of phase one), so budget this as the larger line item — on the order of **$700–$1,400** depending on the exact per-epoch time at 1024 px. This is the step that buys the sharp, photoreal boundaries.
- **Run the finishing passes** (calibration, per-subcategory thresholds, TTA, the held-out test eval). Near-free, and it gives you the first real headline test number to quote.

Tier 2 — Squeeze the trained model harder (modest effort, real gains).

- **Stochastic Weight Averaging / checkpoint ensembling.** The tooling already exists (`swa_average.py`); averaging the last several checkpoints typically adds a point or two of IoU for essentially zero extra training.
- **Two-model ensemble at inference** for the hardest customer photos (the code already supports probability-averaging two checkpoints) — sharper boundaries at ~2× inference cost, usable selectively.

Tier 3 — Improve the data (highest ceiling, most effort).

- **More hard data where the model is weakest.** The per-subcategory metrics already pinpoint the weak spots — generic/ambiguous fences, damaged construction, harsh lighting, complex backgrounds. Targeted scraping + labelling of a few thousand more images in exactly those buckets raises the ceiling more than any architecture change.
- **A second human-review pass** on the lowest-confidence training masks (the project already maintains a "suspect samples" worklist of near-empty masks). Cleaner labels lift every model trained on them.
- **Decide the licensing posture.** If the product needs a fully clean commercial provenance, retrain on the 15,886-image SAFE subset. It's smaller, so expect a modest accuracy trade-off — but it removes all licensing ambiguity. This is a business decision the data is already prepared for.

Tier 4 — Bigger swings (optional, diminishing returns).

- **Train the H+ backbone.** The larger DINOv3 H+ (~840M) backbone is already plumbed in as an option. It would likely add accuracy at materially higher training and serving cost (it wouldn't fit on the cheap L4/T4 GPUs). Worth it only if Tiers 1–3 are exhausted and you want the absolute frontier.
- **Distil to a smaller, faster model** once the big model is finished, to cut serving cost — the opposite optimisation, for when quality is "done" and you want it cheaper.

The recommendation in one line: do Tier 1 first (finish phase one, run phase two, run the finishing passes), and in parallel give the cleaning pipeline the dedicated attention described below. That is the highest-confidence path from today's "staining is almost there, cleaning isn't yet" to "both are genuinely best-in-class," it reuses everything already built, and the training portion costs roughly **$1,200–$2,000** of additional GPU time rather than a new project.

Fixing the cleaning result specifically

Because the cleaning result isn't where you want it yet, it deserves its own plan — and the important insight is that **cleaning quality has two independent levers**, only one of which is "train the model more":

1. **Better masks (the model lever).** Both Apply Stain and Clean Fence paint only inside the fence mask. A sharper, more accurate mask means the cleaning effect stops exactly at the plank edges instead of bleeding onto grass, posts, or background — so finishing training (Tiers 1–3 above) directly improves cleaning too. This is the shared foundation.
2. **The cleaning algorithm itself (the image-processing lever).** "Clean Fence" is a *separate* client-side algorithm from the model — a frequency-preserving wood restoration (an edge-aware guided filter that removes dirt/algae, a finer filter that preserves and re-sharpens the grain, per-plank colour variation, and a brightness lift toward a derived "clean wood" reference). If the current output looks too flat, too uniform, too washed-out, or not "clean enough," that is tuned in the algorithm — independently of the model — by adjusting the reference-colour estimation, the brightness-lift cap, the chroma ceiling, the grain-preservation strength, and the guided-filter smoothing. This is faster to iterate than training (it's a parameter-tuning loop on real photos, not a multi-day GPU run) and is where most of the near-term cleaning improvement will come from.

The concrete recommendation for cleaning: **(a)** treat it as its own short tuning project against a set of your real customer photos — adjust the restoration parameters until weathered, mossy, and grey fences come back looking convincingly fresh without going flat or fake; **(b)** let the ongoing model training improve the mask underneath it; and **(c)** if needed, consider a more advanced cleaning approach (for example, a dedicated learned "de-weathering" step) only after the parameter tuning is exhausted. Most of the gap is almost certainly closable with (a) plus the mask improvements from (b), without a new model.

17. Hosting: what 24/7-warm costs, and the paths to choose

Today the model runs on Google Cloud Run in **scale-to-zero** mode: near-zero cost at idle, but a 30–60-second cold start on the first request after a quiet period. You asked what it costs to keep it warm around the clock, and what the alternatives are. Here is the honest analysis. (All dollar figures are *estimates* from current public GPU pricing and should be re-confirmed against live rates before you commit — the repo defines the machine spec but not the prices.)

The current spec being priced

1× NVIDIA L4 GPU, 4 vCPU, 16 GiB RAM, CPU always allocated, in us-central1.

Option A — Keep it warm 24/7 on Cloud Run (`min-instances 1`)

This pins one L4 instance alive all the time, eliminating cold starts entirely. Cloud Run bills the GPU, vCPU, and memory per second for the full 730 hours in a month. An L4 runs roughly **$0.71 per GPU-hour**, so the GPU alone is about **$510/month**, and adding the always-allocated 4 vCPU + 16 GiB on top brings the realistic all-in figure to roughly:

> **~$510–$700 per month** to keep the model warm 24/7 (the lower end assumes committed-use / sustained-use discounts on the GPU; the GPU by itself is already ~$510).

What you get: every visitor, every time, gets a sub-second-to-few-second response with no "starting up" wait. **What you pay for:** an idle GPU overnight and on slow days.

Option B — Stay scale-to-zero (the current setup)

> **Realistically under $5–$30/month** at low-to-moderate traffic (hundreds to low-thousands of detections), plus near-zero idle cost.

What you get: you pay almost nothing except when the tool is actually used. **What you pay for:** the first visitor after each quiet period waits 30–60 seconds. The browser's wake-on-page-load ping already hides much of this for anyone who lands on the page before clicking.

Option C — A cheaper serverless GPU (Modal on T4)

The project already has a working **Modal** deployment on a cheaper **T4** GPU, also scale-to-zero. Modal is a reasonable middle path: a warm CPU container is ~$10/month, but a warm *GPU* container is materially more (low hundreds). Left scale-to-zero it behaves like Option B on a cheaper card. This is a good "keep in your back pocket" alternative and a sensible place to run a second region or a failover.

Option D — A dedicated cloud GPU VM

Renting a dedicated L4 or A10 virtual machine (Google Compute Engine, RunPod, Lambda, etc.) lands in a **broadly comparable ~$300–$600/month** range — a self-managed VM can run a bit cheaper than a serverless-warm instance — and trades the serverless convenience for you (or us) managing the box, the updates, and the uptime. Generally only worth it if you also have other GPU workloads to amortise on the same machine.

Option E — Buy/build your own small GPU server

For a single-tenant marketing tool on your own site, owning the hardware is genuinely competitive:

	Used RTX 3090 (24 GB) box	New RTX 4090 build
One-time hardware	~$1,000–$1,800	~$2,500–$3,500
Electricity (continuous)	~$15–$40/month	~$15–$40/month
Fits the 2.45 GB model?	Yes, comfortably	Yes, comfortably

The crossover maths is striking: against a ~$550/month warm-cloud bill, a self-built box **pays for itself in roughly 3–6 months** and then costs only electricity. The trade-offs are the usual ones of owning infrastructure: you're responsible for uptime, networking, security, and a public-facing endpoint, and there's no automatic scaling if you ever get a sudden traffic spike (a press mention, a viral post).

The recommendation

Situation	Recommended host
Today's traffic, cost-sensitive	**Stay scale-to-zero (Option B)** — accept the occasional cold start
Cold start is hurting conversions	**Warm Cloud Run (Option A)**, ~$510–$700/mo, or…
Steady, predictable traffic long-term	**Own a 4090/3090 box (Option E)** — cheapest over a year+
Need a cheap failover / second region	**Modal on T4 (Option C)**

The pragmatic call: **keep scale-to-zero as the default**, and only move to a warm instance (or your own box) when cold-start latency is demonstrably costing you customers. The cleanest "best of both" is often to run scale-to-zero plus a tiny scheduled "keep-alive" ping every few minutes during business hours, which keeps the instance warm when customers are actually around without paying for an idle GPU at 3 a.m. — a middle option worth considering if the binary warm/cold choice feels unsatisfying.

18. A single photo's journey, end to end

To tie the whole system together, here is exactly what happens — in order — from the moment a customer drops a photo onto the page to the moment they download a stained preview. Every step in this list is something described in detail earlier; this is the assembled picture.

1. **The page is already warming the engines.** When the customer first loads the page, two things start in the background before they do anything: the browser pings the Cloud Run server's health endpoint so the GPU container begins waking up, and it quietly downloads the small on-device "fence gate" model so it's ready.
2. **The customer picks a photo.** A 4000×3000 phone photo, say. The app keeps the full-resolution original in memory for the final composite, and immediately rejects anything over 10 MB.
3. **The fence gate checks it on the device.** Before anything is uploaded, the OWL-ViT gate (running in a background thread) looks at a 768-pixel copy and asks "is there a wooden fence here?" If the customer accidentally uploaded a selfie or a photo of their dog, they get an instant, polite "no fence detected" message and *nothing is sent to the server* — saving a paid GPU call. If a fence is plausibly present (or if the gate errors, since it fails open), the flow continues.
4. **The photo is prepared for upload.** The app downscales the image to 1024 pixels on its longest side, applies a mild contrast/saturation/auto-levels enhancement to help detection, and encodes it as an 85%-quality JPEG — typically 150–400 KB.
5. **The server finds the fence.** That JPEG is POSTed to the Cloud Run `/detect` endpoint. On the GPU, the image is resized to exactly 512×512, normalised, and run through the full DINOv3 model (backbone → ViT-Adapter → pixel decoder → Mask2Former → refinement head with depth). The server sends back a 512×512 grayscale PNG (~30–100 KB) where each pixel's brightness is the model's confidence that it's fence. If the container was cold, this first call takes 30–60 seconds while it boots; otherwise it's a second or two.
6. **The browser turns confidence into a clean mask.** The PNG is decoded, the confidence values are soft-thresholded into a feathered alpha, upscaled to working resolution, and run through the ~15-filter cleanup gauntlet — guided recovery of shadowed planks, then removal of vegetation, sky, bark, trunks, brick, off-colour blobs, and background buildings, then hole-fill and bottom-extend. If the first result was weak, the cascade kicks in (looser re-reads, one aggressive re-upload, tiled multi-scale detection). All the heavy maths runs in WebAssembly (or WebGPU/JS as fallback).
7. **The stain is applied.** The customer's chosen colour is blended into the fence in "smart" mode — keeping the stain's hue but letting lightness and saturation follow the real wood grain — at the customer's opacity. Because this is all local, switching colours or opacity re-renders instantly with no further server calls.
8. **Or the fence is cleaned.** Alternatively, "Clean Fence" runs the frequency-preserving wood-restoration algorithm to show the existing wood pressure-washed and refreshed (the step currently being improved).
9. **The customer downloads the result** as a high-quality JPEG, named for what they made (stained, cleaned, or both).

The division of labour is the whole design philosophy in one sentence: **the expensive, occasional, "find the fence" step runs once on a GPU server; everything fast, interactive, and private runs on the customer's own device.**

19. Risks, caveats, and recommended fixes

In the spirit of an honest engineering report, here is a consolidated register of the things worth knowing — most are minor, none are blockers, and each has a clear fix. These were surfaced by reading the project's own files; none changes the headline story, but a diligent client should see them in one place.

#	Item	Impact	Recommended fix
1	**No held-out test metric yet** — training paused before the final test/calibration hook ran.	All quoted accuracy is validation, not test.	Finish (or briefly resume to a checkpoint and run) the test-eval pass to get a quotable headline number.
2	**Cleaning result not yet satisfactory.**	Customer-visible.	Dedicated parameter-tuning of the cleaning algorithm + mask improvements from training (see §16).
3	**Deployed (epoch-24) ONNX never passed its end-to-end self-check** (it ran out of memory on the laptop doing the check; the earlier epoch-18 export passed cleanly and the architecture is identical).	Low — almost certainly fine.	Re-run the parity check once on a machine with more memory.
4	**Licensing:** ~52.5% of the training corpus is from sources with ambiguous reuse rights.	Commercial/legal.	The licence-clean 15,886-image subset already exists; decide whether to retrain the production model on it (§3, §16).
5	**A latent "wrong backbone" footgun:** one code file *defaults* to the larger H+ backbone, even though every real config uses ViT-L.	Could cause an accidental wrong-model training run.	Change the code default to ViT-L so the safe choice is the default.
6	**Stale internal documents:** the dataset datasheet, the web README, and the training guide still describe earlier versions (pre-review counts, the old in-browser model, "H+").	Could mislead a future reader.	Refresh or clearly mark these as historical before they're shared.
7	**Cosmetic naming drift in code:** the browser's server constant is still called `MODAL_ENDPOINT` (it's Cloud Run now), and some gate comments say "OWLv2/SigLIP" (it's OWL-ViT now).	None functional.	A quick rename/comment cleanup for future maintainers.
8	**Dead-weight files:** leftover SigLIP "baked features" from the gate's previous design are still copied into the deployment bundle.	Slightly larger download.	Delete them from the packaged plugin.
9	**Minor data-count drift** on the positive set (catalogue 21,674 vs. database 21,518 vs. disk 21,414).	None — disk is authoritative.	Optional: re-sync the catalogue to disk.

None of these is urgent. Items 1, 2, and 4 are the ones that actually matter for the product and the business, and all three are addressed in the roadmap.

20. Appendix A — Technology stack at a glance

Layer	Technology
Data collection	Python, asyncio, httpx, Playwright; SQLite state; Google Vision QA
Data refining	Python, Pillow, parallel integrity check; SHA-256 + perceptual-hash dedup
Labelling	Grounding DINO (detect) + SAM 2.1 (segment) + CLIP (scene filter) + SAM 3 (manual)
Model	PyTorch; DINOv3 ViT-L/16 + ViT-Adapter + Mask2Former + UNet3+ refinement + MiDaS depth
Training	vast.ai (1× A100-SXM4-80GB, 128 GB RAM, Xeon Platinum); AdamW, bf16, EMA
Export	ONNX (opset 17→18), external-data weights, baked calibration, parity check
Serving	Google Cloud Run (L4 GPU) — live; Modal (T4 GPU) — sibling/fallback; FastAPI + ONNX Runtime
Frontend	Single-page HTML + WordPress plugin (Shadow DOM); transformers.js; WebAssembly (SIMD) + WebGPU
On-device gate	OWL-ViT zero-shot object detection in a Web Worker
Hosting (frontend)	HuggingFace Spaces (static) / GitHub Pages / Cloudflare; WordPress at ninjafencestaining.com

21. Appendix B — Key numbers, in one place

Metric	Value
Images scraped (positive / negative)	21,414 / 12,009
Master catalogue size	33,423
Cross-set duplicates removed	251
Class balance after human review	13,328 fence / 20,095 not-fence (~8,000 corrected)
Train / val / test split	23,394 / 5,013 / 5,016
High-res (phase-2) split	14,529 / 3,087 / 3,084
Pixel-mask answer keys	33,423
Golden-set benchmark	100 hand-masked images
Licence breakdown	SAFE 37% / SAFE-attrib 10.5% / RISKY 52.5%
Model	DINOv3 ViT-L/16, ~485M trainable params
Training spend	$220 @ $1.14/GPU-hr ≈ 193 GPU-hours (~8 days)
Phase-1 progress	24 / 120 epochs (20%)
Best validation IoU (epoch 24)	0.5014 (still climbing)
Phase-2 progress	0 / 70 epochs (not started)
Deployed model files	13.9 MB graph + 2.45 GB weights
Production host	Google Cloud Run, 1× NVIDIA L4, scale-to-zero
Cold start	~30–60 s; warm response sub-second to ~2 s
24/7-warm cost estimate	~$510–$700/month
Browser stain colours	19 (across 3 families)

22. Appendix C — Glossary

- **Segmentation** — deciding, for every pixel, what category it belongs to (here: fence vs. not-fence). The output is a "mask."
- **IoU (Intersection-over-Union)** — the standard segmentation accuracy score: of all pixels that are either truly fence or predicted fence, the fraction that are both. 1.0 is perfect.
- **Backbone** — the pre-trained "eyes" of the model that turn pixels into rich features (here, DINOv3 ViT-L/16).
- **ViT (Vision Transformer)** — a modern neural-network design that processes an image as a grid of patches.
- **DINOv3** — Meta's self-supervised pre-trained vision model, trained on 1.7 billion images without human labels.
- **ViT-Adapter** — an add-on that gives a Vision Transformer the fine, multi-scale detail needed for pixel-accurate masks.
- **Mask2Former** — a modern segmentation decoder design using learnable "queries" to produce masks.
- **Refinement head** — the final network stage that sharpens the mask's boundaries.
- **MiDaS / DPT** — a pre-trained model that estimates depth (how far away each pixel is) from a single photo; used here as a fixed "teacher."
- **EMA (Exponential Moving Average)** — a smoothed running-average copy of the model's weights that generalises better; it's what gets deployed.
- **ONNX** — an open, portable model format that any inference engine can run.
- **Epoch** — one full pass of the model over the entire training set.
- **Augmentation** — randomly perturbing training images so the model learns the concept, not the specific pixels.
- **Cold start** — the delay when a scaled-to-zero server has to boot up for the first request after being idle.
- **Zero-shot** — a model that can handle categories described in plain text without being specifically trained on them (used by the on-device fence gate).
- **WebAssembly / WebGPU** — browser technologies for running fast, near-native code (WASM) and GPU computation (WebGPU) on the user's own device.

Prepared by TechnoTaau. Every figure in this report was read directly from the project's own configuration files, training logs, model metadata, and deployment scripts as of June 2026. Where the project's internal documents disagree with each other, this report uses the most authoritative on-disk source and notes the discrepancy. Cost figures for third-party GPU hosting are estimates from public pricing and should be re-confirmed against live rates before any commitment.